



Navigating the Landscape of Automated Feedback Generation Techniques for Programming Exercises

SMITHA S. KUMAR, School of Mathematical and Computer Sciences, Heriot-Watt University,
Dubai, United Arab Emirates

MICHAEL A. LONES and MANUEL MAAREK, Heriot-Watt University, Edinburgh, United Kingdom

HIND ZANTOUT, School of Mathematical and Computer Sciences, Heriot-Watt University,
Dubai, United Arab Emirates

Programming demands a variety of cognitive skills, and mastering these competencies is essential for success in computer science education. The importance of formative feedback is well acknowledged in programming education, and thus, a diverse range of techniques has been proposed to generate and enhance formative feedback for programming exercises. This article reviews state-of-the-art automated feedback generation techniques and categorizes the various approaches based on the underlying computational techniques, programming languages, the kind of programming errors they deal with, and the type of feedback they provide. It covers data-driven techniques, those which use program repair methods, machine learning-based techniques, and techniques based around the use of large language models, particularly noting the rapid uptake of the latter. The article provides a summary of key findings and challenges, alongside recommendations for future work. The findings reveal that although there exist numerous tools for automated programming feedback, many studies depend on non-public benchmarks, which limits reproducibility and independent evaluation of the tools and their datasets. Additionally, tools are not always language agnostic and in some cases involve complex configuration steps. Large language models have demonstrated transformative potential in generating feedback. However, most research has focused on introductory courses (CS1 and CS2) indicating the need to apply them in advanced fields like machine learning and image processing. Although large language models have outperformed traditional approaches, challenges related to hallucinations and incorrect responses still need to be addressed as precision is critical in a pedagogical setting. Most of the studies use proprietary models that lack transparency and customization options, emphasizing the need for further research into open LLM alternatives.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Applied computing** → **Computer-assisted instruction**;

Additional Key Words and Phrases: literature review, automated feedback, hints, learning programming, programming languages

Authors' Contact Information: Smitha S. Kumar (corresponding author), School of Mathematical and Computer Sciences, Heriot-Watt University, Dubai, United Arab Emirates; e-mail: sd102@hw.ac.uk; Michael A. Lones, Heriot-Watt University, Edinburgh, United Kingdom; e-mail: M.Lones@hw.ac.uk; Manuel Maarek, Heriot-Watt University, Edinburgh, United Kingdom; e-mail: M.Maarek@hw.ac.uk; Hind Zantout, School of Mathematical and Computer Sciences, Heriot-Watt University, Dubai, United Arab Emirates; e-mail: H.Zantout@hw.ac.uk.



This work is licensed under Creative Commons Attribution-NonCommercial-NoDerivatives International 4.0.

© 2025 Copyright held by the owner/author(s).

ACM 1946-6226/2025/10-ART54

<https://doi.org/10.1145/3764593>

ACM Reference format:

Smitha S. Kumar, Michael A. Lones, Manuel Maarek, and Hind Zantout. 2025. Navigating the Landscape of Automated Feedback Generation Techniques for Programming Exercises. *ACM Trans. Comput. Educ.* 25, 4, Article 54 (October 2025), 29 pages.
<https://doi.org/10.1145/3764593>

1 Introduction

The recent technological developments in AI, machine learning, and cybersecurity ensure that there will be a high demand for programmers who can design and maintain quality software systems. Thus, excellent programming skills will continue to be one of the key competencies for employment across a number of IT sectors [104]. However, programming continues to be a major challenge for many students in their introductory programming course at university (commonly called CS1) [9, 109]. Numerous research studies have attempted to characterize the mistakes committed by novice programmers and emphasized the need for clear, concise feedback [21, 27, 44, 69, 70]. Due to the significant number of students enrolled in CS1, generating personalized feedback requires a substantial amount of work from the instructor and may be impractical [7]. Moreover, the vast solution space of open-ended programming exercises (i.e., those where there are multiple solution strategies) further compounds the challenge. Therefore, researchers have proposed various techniques to automate the feedback generation process. According to Narciss [75], “feedback refers to all post-response information which informs the learner on his/her actual state of learning or performance in order to regulate the further process of learning.” There are various types of feedback, such as knowledge of performance (summative feedback on performance, such as “10 of 20 correct” and “50% correct task”), **knowledge of result or response (KR)** (correct, incorrect), and **knowledge of the correct results (KCR)** (shows the expected solution). However, these forms of feedback provide limited guidance to the learner for improving the program’s solution state. Elaborated feedback types include knowledge about task constraints, knowledge about concepts, **knowledge about mistakes (KM)**, **knowledge about how to proceed (KH)**, and knowledge about meta-cognition, and these types of feedback can be more useful for the learner.

This article reviews research on automated feedback generation techniques for programming exercises. There have been a number of previous surveys in both this area [12, 48, 68] and in the related area of automated assessment [73, 81]. Keuning et al. [48] reviewed the various **automated tutoring systems (ATS)** published until 2015 and introduced a classification system based on the feedback components by Narciss [75]. They classified the tools based on the feedback generation techniques, adaptability of the tools for creating personalized feedback, and the evaluation criteria used to access the tools. Cavalcanti et al. [12] conducted a survey of papers published from 2009 to 2018 and examined the effects of automated feedback on student performance and the workload of the instructors. Although the review is not specific to feedback on programming, the findings indicate that automated feedback improved student performance. There was insufficient evidence to claim that it reduced the instructor workload. They observed that the most widely used techniques for feedback generation were pattern matching, rule-based systems, machine learning, and **natural language processing (NLP)**. The findings also emphasized the need for developing tools for instructors and exploring effective techniques to assess the feedback quality. McBroom et al. [68] reviewed the papers published from 2014 to 2018 and integrated the various techniques into a new framework called **hint iteration by narrow-down and transformation steps (HINTS)**. The study emphasized that hint generation techniques share common foundational elements and that identifying these similarities, analyzing the nature of the hints, and designing appropriate evaluation methods can help improve hint generation systems. While Keuning et al. [48] examined

the various AI-based feedback generation techniques, it provides limited insights into the latest developments. One limitation of the study by Cavalcanti et al. [12] is that it did not provide sufficient detail about the feedback generation approaches used in the reviewed studies. Similarly, McBroom et al. [68] presented some of the automatic program repair-based techniques; however, they did not present a comprehensive analysis of the various techniques.

Messer et al. [73] surveyed **automated assessment tools (AATs)** and classified them based on the programming skills assessed, feedback generation techniques, supported programming language, degree of automation, and the techniques used for evaluation of the tools. The paper emphasized that most AATs focused on the correctness of object-oriented programming assignments using static and dynamic analysis techniques, and the feedback generated was often limited to a pass or fail report based on unit testing. Most of the tools focused on object-oriented programming languages, and the tool performance was evaluated using surveys or comparisons with human graders. Another study by Paiva et al. [81] surveyed various AATs, focusing on types of exercises supported, security of code execution, testing techniques, and feedback elements. They identified a shift toward more comprehensive evaluation criteria that not only include functional correctness but also program efficiency, behavior, and readability. A potential limitation of their study was the selection process, where 15 tools were selected based on relevance, and 15 were randomly selected. This might have resulted in exclusion of some significant tools. Although the focus of these studies was on AATs, their analysis of the feedback element reveals the shortcoming of the feedback currently offered by AATs. Nevertheless, these reviews provide valuable insights for the research community and highlight research gaps that remain unexplored.

In this context, we surveyed the various tools and techniques published on automated feedback generation for programming exercises during the period 2017–2023. Our review focuses on the following:

- Surveying the current landscape of automated feedback generation techniques, including LLM-based techniques.
- Categorizing feedback generation techniques based on the underlying computational technique, error type, feedback type, and evaluation criteria.
- Analyzing the shortcomings and gaps in existing research.

This article is organized as follows: In the following section, we discuss the subject of feedback generation in the context of programming education. Section 3 outlines the research methodology. Recent automated feedback generation techniques are discussed in Section 4, followed by a discussion of the findings of this survey in Section 5. Finally, conclusions and recommendations for future work are covered in Section 6.

2 Background

This section covers the various challenges in introductory programming education (CS1) and provides a detailed overview of the evolution of automated feedback generation systems.

2.1 Challenges in Programming Pedagogy

There have been numerous research studies done in the past to identify the key challenges faced by novice programmers. Becker and Quille [8] analyzed 481 CS1 papers published at the SIGCSE Technical Symposium over the past 50 years and provided an overview of the evolution of programming education research. Their findings indicate a growing emphasis on inclusivity and student-centered issues such as diverse learning styles, cognitive barriers and error-handling skills in programming education.

In another study, Luxton-Reilly et al. [63] classified research papers into four categories: student-related issues, pedagogical methods, curriculum planning, and assessment design. The key challenges include addressing student misconceptions, supporting diverse learners, fostering engagement, conducting effective assessments with timely feedback, and updating the curriculum with emerging topics. Denny et al. [20] identified programming pedagogy, student behavior, and student understanding as the top three research themes valued by computing educators.

Robins [91] investigated the various challenges associated with training novice programmers. High dropout and failure rates, limited conceptual understanding of concepts, and bimodal grade distributions were identified as the key challenges in CS1 courses. The paper further emphasized that programming involves cognitive competencies such as problem analysis, planning, problem-solving, and debugging, rather than merely mastering languages and tools. In another study, Goldman et al. [30] found that procedural concepts (e.g., parameter scope, procedure design), object-oriented programming concepts (inheritance, polymorphism, and primitives vs. reference variables), algorithm design concepts (abstraction, recursion, memory model, references, pointers), and program design concepts (modularization, debugging, designing solutions, designing tests) were the most important and challenging topics in programming fundamentals. These insights can assist educators in prioritizing certain topics during instruction. Medeiros et al. [72] surveyed papers from 2010 to 2016 on introductory programming in higher education and identified that novice programmers face challenges in problem-solving, understanding abstract programming concepts, and algorithmic thinking. Additionally, their learning is further hampered by poor motivation, lack of confidence, and ineffective time management.

The findings from these studies emphasize the importance of meaningful feedback in enhancing students' academic experience. The rapid growth of enrollments in programming courses has driven the transition from manual to automated feedback systems [73].

2.2 Automated Feedback

The evolution of automated feedback generation systems began in the 1960s, and the first-generation systems focused on workflow automation to save the time and effort of human graders. The programming exercises in these systems were mostly multiple-choice questions. Second-generation systems focused on designing innovative strategies to assess the quality and correctness of student submissions. The availability of a large volume of data during the mid-2000s led to third-generation systems focusing on enhancing feedback quality [25]. Feedback generation has typically been a two-step process: identify the fault in the code and later generate a suggestion to fix the issue. One of the major challenges in creating feedback for programming assignments is the diverse range of possible solutions. The generated feedback consisted of formative and summative types. While summative feedback reflects the overall performance, formative feedback guides the learners to achieve the learning objectives [37]. The various types of automated feedback that students may receive are correctness feedback, code quality feedback, program repair feedback, next-step hints, example feedback, error-specific feedback, peer feedback, and teacher feedback [36]. Correctness feedback provides information about whether a student's solution is functionally correct or not. Nevertheless, this information might not help the learner fix the issue in the code. Other types of feedback, such as program repair feedback, error-specific feedback, and next-step hints, contain more useful information. Multiple research studies highlight the need to create systems that can generate positive and timely feedback [66].

2.3 AATs

Automated grading systems that assess student submissions have been in existence for over 60 years. These systems have evolved from their initial complex versions to more user-friendly web-based

systems [22]. However, many of these systems only generate a grade as the output, although certain web-based systems like Web-CAT [26] do include a hint to troubleshoot the failed test. Accordingly, Lee [55] created a system that unlocks hints based on certain pre-defined conditions to encourage incremental development. Ihantola et al. [42] reviewed the various AATs developed between 2006 and 2010 and identified the major features supported by those systems. These include support for test definitions and resubmission policies, multiple programming languages, security and sandboxing, integration with learning management systems, a mix of manual and automated feedback, and tools tailored to the nature of the task. They also suggested the need to adopt a collaborative approach to improve the existing systems to build robust AATs. In another study, Lajis et al. [51] observed that AATs lacked standardized grading systems aligned with the Bloom's Taxonomy to accurately assess the programming skills. They argued that since programming demands a variety of cognitive skills varying from basic comprehension to higher-order thinking, it is essential to develop tools that not only assess the correctness of the code but also measure the problem-solving abilities of the student. Romli et al. [93] reviewed the various approaches used in automated programming assessments and the test data generation techniques used in the software testing field in 2016. While dynamic testing is widely used for automated assessments, the available test data generation techniques are often underutilized to support this process. Therefore, this study emphasizes the need to integrate these test data generation techniques into an automated programming assessment system to improve the programming assessment process.

Souza et al. [99] conducted a systematic literature review and identified 30 assessment tools and proposed a new classification scheme based on assessment type, approach, and speciality in 2016. They also noted a trend toward assessment tools that automate the process and deliver immediate feedback to the learners. Although instructors have access to a variety of tools, choosing the right tool remains a challenge. As a result, the authors propose integrating AATs into learning management systems and offering structured guidance to support instructors.

In another review conducted by Ismail and Lakulu [43], several AATs published from 2014 to 2019 were classified based on the following criteria: evaluation methodology, key features, supported languages, assessment strategy, and pedagogical elements such as rubrics and learning taxonomies. It was observed that although the tools demonstrated technical strength, they fell short in incorporating the pedagogical features like rubrics and learning taxonomies. The authors emphasize that addressing these aspects is crucial for AAT adaptability in an educational setting.

2.4 Overview of Feedback Generation Approaches

In a study conducted by Barbosa Rocha et al. [92], six approaches are identified for feedback generation: testcase-based, example-based, corrections for solutions, similar solutions, guides to solutions, errors, and hit identification. Le [52] reviewed several AI-supported tutoring approaches and classified them into example-based, simulation-based, collaboration-based, dialogue-based, program analysis-based, and feedback-based approaches. In an example-based approach, students are provided with example problems and their corresponding solutions and later assigned to solve similar problems. Novice programmers find the program dynamics challenging, and simulation-based systems help them visualize the program behavior. In program analysis-based systems, student programs are analyzed, and appropriate explanations and corrections are generated as feedback. Dialogue-based systems initiate a conversation to guide the student to refactor the incorrect version. Model tracing and machine learning are used in the feedback-based approaches to analyze the student solution and perform appropriate error diagnostics. Unlike the other methods, the collaboration-based approach helps students learn in a group environment and uses NLP techniques. Among all these approaches, the feedback-based approach is the most popular, and some of these

systems can identify the student's intention accurately. This will eventually help in generating appropriate adaptive feedback.

An **intelligent tutoring system (ITS)** is a type of computer-assisted learning software that models the cognitive and emotional states of individual learners with the goal of adapting and personalizing instruction [76]. According to Hieu and Syed Mustapha [40], there are five types of ITS for programming, such as curriculum sequencing, intelligent analysis of student solutions, program debugging support, interactive code-writing problem-solving support, and example-based support. LispTutor [15], developed in 1990, was the first ITS for programming. In an experiment conducted to compare the effectiveness of human tutoring, computer tutoring, and no tutoring, VanLehn [105] observed that ITS is as effective as a human tutor. Data-driven systems are used to provide next-step hints. Price et al. [88] evaluated the quality of the six next-step hint generation algorithms and observed that the source check algorithm [87] offered good performance for simple Python programs; however, there were challenges when the student submissions were significantly different from the sample solutions.

Various methods are applied to generate feedback, such as dynamic code analysis using automated testing, program transformation, basic static analysis, constraint-based modeling, intention-based diagnosis, external tools (other than testing tools), model tracing, data-driven ITS, and other AI-based techniques [48]. In model tracing, the student solution steps are compared with a model solution, and feedback is generated if there are discrepancies in the steps. For instance, Ask-Elle [45], a programming tutor for Haskell, supports incremental development of code and generates automated textual feedback for class 3 (open-ended) programming exercises.

A data-driven hint generation approach was first used in Hintfactory [100], where peer data are used for generating the next-step hints. Among the various approaches discussed earlier, automated testing is one of the most widely used. In this method, the test execution results are used to generate feedback [68]. For instance, Autograder [39] uses the JUnit testing framework to evaluate the student submission. Several novel approaches have been proposed in recent years to repair programs written by expert developers and novice programmers and to generate feedback. According to Monperrus [74], "automatic repair is the transformation of an unacceptable behavior of a program execution into an acceptable one according to a specification." Specifications could be test suites, natural language documents, or formal logic formulae. The automatic program repair pipeline consists of three main steps: **fault localization (FL)** (step 1), variant creation (step 2), and variant validation (step 3). The input to this pipeline is incorrect code and the associated test cases, and a plausible patch is generated as the output. Patch generation can be achieved using search-based techniques, semantic analysis, and deep learning approaches [54].

Toll et al. [103] surveyed 27 papers published on ATS from 2009 to 2019 and identified the various challenges in automated tutoring. Most of the ATS systems provide generic hints that do not guide the learner to move from an erroneous program to a correct one. Some of the ATS systems use model-goal solutions to generate feedback; however, this approach suffers from scalability issues. Paiva et al. [80] conducted a study on papers published from 2010 to 2022 and concluded that there is an increase in the number of publications focused on automatic program repair, machine learning, and static code analysis for feedback generation. Considering prior approaches, challenges, and the new developments in AI, our study examines the recent developments in this area. The next section of this article outlines the method we followed in this research study.

3 Methodology

The scope of this research is the recent (2017–2023) developments in automated feedback generation for programming tasks. Our methodology involved formulating the **research question (RQ)**, searching across relevant databases, selecting and filtering studies, and finally performing data

Table 1. Inclusion and Exclusion Criteria

Inclusion	Exclusion
The paper presents a novel technique/tool and assesses its overall effectiveness.	The full text could not be retrieved.
The paper was made public between 2017 and 2023 inclusive (either published or on arXiv).	The paper is a survey of programming education.
The paper focuses on feedback for programming assignments.	The paper is not peer-reviewed (unless on arXiv with citations by the end of 2024).
	The paper includes only the abstract.
	The paper is about a new dataset.
	The paper explores gamification in programming education.
	Posters, Master’s theses, and short papers.

analysis and synthesis. Our RQs are discussed in Section 3.1. The search process for the relevant papers is described in Section 3.2. Inclusion and exclusion criteria are discussed in Section 3.3.

3.1 RQ

The following RQs form the basis of our investigation into automated feedback generation for programming exercises:

- RQ1. Which computational techniques are used to generate feedback for programming exercises?
- RQ2. What types of feedback are generated, and how useful are they for learners?

3.2 Search Process

In this review process, we followed the following steps. We manually reviewed the articles published in ACM TOCE journal to identify domain-relevant keywords related to our RQ to construct the search query. We chose ACM TOCE for its prominence in computer science education and its reputation for publishing impactful, high-quality work. Based on the keywords identified, we formulated the search string (query) below.

“feedback*” OR “hint*” OR “automated feedback” OR “feedback generation” OR “error repair” OR “fault localize”) AND (“programming education” OR “programming assignments”) AND (“students” OR “learners” OR “novice programmers”)

We applied the search string to the ACM Digital Library, IEEE Xplore, and Google Scholar to identify relevant studies. ACM and IEEE were chosen as they have an extensive collection of computer science education-related resources. Google Scholar was chosen to identify any papers beyond the scope of ACM and IEEE. Filters were applied on ACM and IEEE search to include papers published in journals and conferences between 2017 and 2023 inclusive. Google Scholar search was conducted for the same time frame, with the filter set to include all research articles. A total of 10,344 papers were identified in the preliminary search from the three databases.

3.3 Inclusion and Exclusion Criteria

As the initial search returned a substantial number of papers, we applied several exclusion criteria. We excluded papers based on title and abstract screening, non-peer-reviewed publication types (unless on arXiv with citations by the end of 2024), inaccessible full texts, and those outside the scope of programming education. Table 1 summarizes the various inclusion and exclusion criteria.

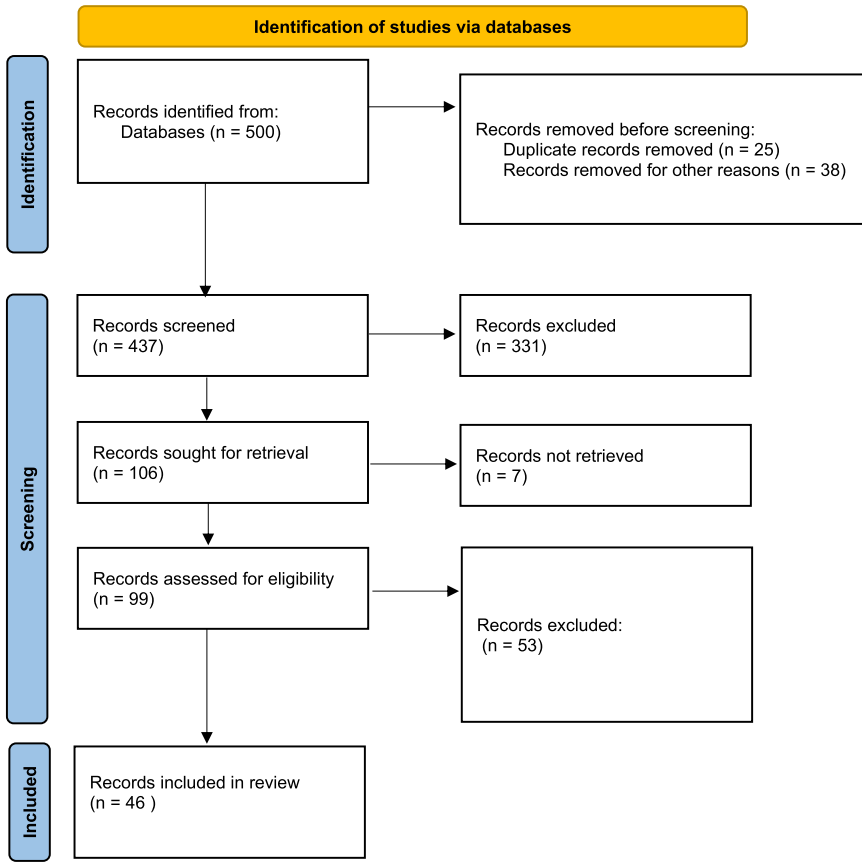


Fig. 1. PRISMA flowchart showing inclusion and exclusion decisions [35].

The preselection process retrieved 500 papers from three databases. We followed the PRISMA reporting guidelines for systematic reviews, as summarized in Figure 1.

A total of 331 papers were excluded in the screening stage based on an assessment of their abstract, introductions, and relevance to the RQ. Seven papers were omitted due to lack of access to the full text. In the full text scan, we reviewed 99 papers and excluded 53 papers. We selected papers that presented a novel tool/approach for automated feedback generation. We found several papers related to the same tool and have included each tool once. Based on the analysis, 46 novel tools/approaches that meet the criteria were shortlisted. We have classified the papers based on multiple criteria, such as the underlying computational techniques, error types, and feedback types. As a result, this study is a comprehensive survey of recent research focused on automated feedback generation for programming exercises. This survey was carried out by one researcher under the supervision of experts in computer science education and machine learning. The categorization of selected papers was carried out based on the feedback generation techniques they utilized. The next section of this article provides a comprehensive review of the most recent approaches for automated feedback generation, critically analyzes the effectiveness of the existing approaches to generate high-quality feedback for programming exercises, and identifies the key challenges that should be addressed.

4 Current State of Automated Feedback Generation

The first RQ in this study sought to determine the various computational techniques used to generate feedback for programming exercises. This section outlines the recent techniques used in feedback generation, systematically categorizing the reviewed papers based on their underlying methodology. The main categories include data-driven techniques, machine learning-based approaches, program repair methods, and **large language models (LLMs)**. Additionally, this section describes the feedback type associated with each study. Table 3 presents an overview of the reviewed papers based on the underlying computational technique, error type, and feedback type.

4.1 Data Driven

This section discusses the various studies that use existing student submissions or expert solutions to generate feedback. This approach was first employed in the Hint Factory [100], a hint generation system for logic tutor. Similarly, Clara [31] extended this idea by clustering correct student submissions for a given programming assignment when they share the same **control-flow graph (CFG)** and exhibit a bijective relation between the variables of both programs (i.e., variables take the same values in the same order during a program execution). The CFG represents all possible paths a program might take during execution. The repair algorithm is run against each cluster, and the syntactic distance is used to compute minimal fixes. Minimal fixes are the fewest changes needed to correct an incorrect program. Clara achieved a repair rate of 97%, and the feedback generated was rated 3.4 out of 5 by students who participated in the user study. In a similar approach called **Search, align, and repair for feedback generation (SARFGEN)** [107], the system first searches for similar correct programs, aligns them with the incorrect submission to identify the discrepancy, and generates fix suggestions. This approach was able to generate useful feedback for 89.7% of the submissions. Although Clara [31] is marginally faster for smaller programs, it does not scale effectively to larger and more complex code. SARFGEN exhibited better performance for large and complex programs. Refactory [41], on the other hand, handles the CFG difference by transforming the correct programs into semantically equivalent versions. It also applies structural mutations to handle the CFG differences. This significantly increases the percentage of incorrect programs with a similar CFG and also results in high repair rate compared to Clara [31]. Moreover, it can generate repairs even when there are only a small number of correct submissions.

The above approaches collect reference programs through manual selection or test-based selection. Verified Repair of Programming Assignments (Verifix) [1] emphasizes the need for rigorous quality assurance of the feedback generation system and uses **control-flow automata (CFA)** structure of both the reference and the student programs. The CFA of the reference program is structurally aligned to the student program to obtain an aligned CFA, from which repairs are generated. Compared to Clara [31], this approach generates minimal repairs and thus preserves the original student submissions while reportedly achieving better repair accuracy.

Intelligent teaching assistant for programming (ITAP) [90] generates next-step hints automatically for Python programming assignments. Initially, the student submission is transformed into an abstract representation (a process referred to as canonicalization), and later, the closest correct submission is identified, and a set of edits needed to reach that state is created. These edits are then shared as next-step hints. A limitation of this approach is that it works only if the input program is syntactically correct. The usability study findings showed that less experienced students favored hints with detailed information. Codechecker [89] also uses a set of student data to generate hints for Python programs and provides insight into the various design choices of a hint generation system. It outlines three design dimensions: how to select hints, how to display them, and how detailed they should be. Bottom-out hints, which explicitly give away the answers,

and pointing-hints that are abstract are both not particularly beneficial for learning. To bridge this gap, Codechecker generates transformation hints that suggest possible edits that help student move closer to the correct solution. The study found that hint generation systems should clearly suggest precise next steps and should also prioritize the hints.

CATNIP [77], the first hint generation system for Scratch, a visual block-based programming language, is built on Zimmerman's and Rupakheti's hint generation algorithm [118]. A candidate solution is chosen based on automated test results, and abstract syntax tree differences are used to construct a hint. The findings from the evaluation revealed that these hints resulted in functional improvement of the code. StepTutor [108] provides additional worked examples, along with the next-step hint to promote the learning experience in Snap, another block-based programming language. A formative evaluation was carried out to understand the student experience, and the comments were positive. Despite the positive comments, a notable drawback of the evaluation is the same sample size. **Adaptive immediate feedback (AIF)** [65] system provides real-time feedback on progress, correctness, and errors using data-driven techniques in a block-based programming environment. Results from an introductory programming course classroom study indicated that AIF improved student performance. In iSnap [67], an extension of the block-based Snap programming environment, supplementary textual explanations were added to the next-step hints to enhance learners' understanding. In a controlled experiment conducted by the authors, it was found that learners who received a textual explanation alongside the hint had a better understanding of the code hint.

4.2 Machine Learning

This section reviews the various machine learning techniques for feedback generation. **Targeted repair of compilation errors (TRACER)** [2] repairs the compilation errors in C programs by localizing the errors, using the insights that single-line edits in most C programming usually occur close to the compiler-reported line. It uses a **recurrent neural network (RNN)** to predict an abstract form of repair, which is then converted into actual code. TRACER was able to generate fixes for 79.27% of compilation errors, and the generated feedback includes the error location, repair suggestions, and the repaired program. DeepFix [33], on the other hand, uses a multi-layered sequence-to-sequence neural network with attention to fix multiple typographic errors in a C program. A multi-layered sequence-to-sequence neural network is a complex architecture that processes inputs through several layers to generate an output, while the attention mechanism enables the model to focus on the most relevant parts of the input. The program is tokenized and fed into a neural network that produces a fix. Its iterative strategy allows it to correct multiple errors in a single program. DeepFix successfully repaired 27% of programs and partially corrected an additional 19%. Gupta et al. [32] proposed a new deep reinforcement learning-based solution (RLAssist) to fix syntax errors that in which an agent learns to mimic editing actions. The agent interacts with the environment (program text) and performs actions to navigate and modify the program to achieve error-free code. A reward system is employed to guide the agent to successfully repair the code. The findings indicate that this approach outperformed existing state-of-the-art techniques such as DeepFix [33]. Despite their ability to generate repairs for syntax errors, these methods are inefficient at generating fixes for certain error types and require a significant amount of data, training time, and resources. Unlike the previous approaches that provide a repaired solution, **Targeted example generation for compilation errors (TEGCER)** [3] offers closest example solutions. During the training phase the neural network takes erroneous lines as input and predicts their corresponding error-repair class. This classifier is employed to determine the error-repair class of a newly encountered code error and the system retrieves examples that belong to that predicted class. The results of the usability study indicate that students who used TEGCER were able to fix far more incorrect programs than those who did not. Annette [18] is an intelligent tutoring companion

embedded in the Eclipse IDE to support students in an introductory Java programming course. Annete employs a neural network that processes input features extracted from source code and IDE-identified events such as “loop found,” and “keyword double found.” The normalized input data are passed to the neural network, which generates a numerical response that is used to retrieve the corresponding feedback message. In a user study conducted with 28 computer science students, participants reported a positive experience while using the system. A more recent technique called **Modular accelerated compilation error repair (MACER)** [14] localizes a bug and converts the buggy statement into an abstract representation. This abstract representation is used to create a high-dimensional feature vector for predicting the repair class. MACER further predicts the exact location within the source code where the modification should be applied and converts the abstract representation into code. The authors claim that their method outperforms previous approaches and achieves a significant improvement in training time and resource utilization. The MACER framework was further improved to create Caper [34] to fix syntax errors in C programs. This approach required less training time and fewer resources and achieved higher repair accuracy than MACER. In addition, a Python extension called PyPer [34] was also created by the authors. rDSA [106] is a system designed to support students in completing their data science assignments in R. First, student submissions are parsed to extract the comments, which are processed by a NLP module. The extracted features are used by a classifier to verify the correctness of these comments. Finally, a distance calculator module computes the difference between the student submission and the reference solution. The system uses these analysis results to generate detailed feedback. The findings of this study indicated that the tool contributes positively to student engagement and learning outcomes. **Program analytics-based localization oracle (PABLO)** [16] uses a novel approach to localize bugs in Python programs. It extracts the static, dynamic, contextual, and slicing features from the program to train a machine learning model. The model is then used to localize bugs in a new python program. User studies indicate that PABLO was able to successfully localize the bug and generate useful debugging hints. Pyrite is an extension of PABLO that uses a machine learning-based model to repair the bugs in the Python code and generate debugging hints.

Rite [94] has been trained on a set of 4,500 incorrect Ocaml programs along with their correct versions. The system learns the repair pattern by analyzing pairs of erroneous and corrected programs and represents the fixes as abstract syntax trees. A classifier is trained to predict the correct template for an incorrect program. In the final step, the template is transformed to a usable code snippet. Rite accurately predicts a fix, and the user study confirms that the repair quality was higher than the state-of-the-art repair tools. Sensibility [95] presents a novel approach using language models to generate a fix for syntax errors in Java code. It uses an n-gram representation and a Long Short-Term Memory model (a kind of RNN used to process sequence data) to locate errors and generate a fix. This approach differs from earlier techniques in that it uses code collected from GitHub repositories.

4.3 Program Repair

This section presents the various program repair techniques for feedback generation. GradeIT [84], a grading system based on program repair techniques, addresses 55 types of C programming syntax errors and provides simpler error messages. The findings from the user study indicate that GradeIT’s feedback received an average rating of 2.426 out of 3 for usefulness. Nonetheless, it was limited to addressing the 55 error types. In another study [85], a system consisting of a structural checker, a repair synthesizer, and a case analyzer was deployed in an introductory Scheme programming course. The structural checker identifies syntax errors and provides hint, whereas the repair synthesizer uses a mutation-based approach to check programs that are nearly correct. The case analyzer detects missing branches and suggests repairs to the student. While

the findings indicate that hints contributed to the student progress, the paper does not provide a detailed discussion of the user study. IPT [58] uses an improved version of Clara [31] combined with a student cognitive model to predict user performance accurately. The APR techniques will repair the code and identify the type of errors in the submission. This information acts as an indicator to predict user performance. Yi et al. [115] conducted a study using four state-of-the-art APR tools (GenProg [53], AE [110], Angelix [71], and Prophet [61]) to repair bugs in C programming assignments. Even though student programs are simpler than those written by professionals, the repair rate was low (31%). The paper also proposes a novel repair policy to generate hints. The buggy program is run against the test suite to identify the positive and negative tests. The various types of hints include control-flow hints, dataflow hints, and conditional dataflow hints. The algorithm searches for a control-flow hint or conditional-dataflow hint in parallel and terminates when a repair is found or a null value is returned. To identify a control-flow hint, the system runs two APR tools in parallel (Angelix and Prophet) that can modify the conditional expressions of the if/loop statements. In the case of a conditional dataflow hint, a two-step approach is used. In the first step, the four APR tools are run in parallel (GenProg, AE, Prophet, and Angelix), and Pb (student program which fails one or more test cases) is modified either by adding, deleting, or modifying the statement to create a refactored program (Pi). Pi is further refined using Prophet and Angelix to achieve a positive result for the Tp (positive test cases). These steps are iteratively repeated until all the bugs are fixed. Although the graders found the repair suggestions useful, it was observed that the novice programmers were unable to effectively use the hint.

Cafe [98] uses a context-aware repair algorithm that can repair OCaml programs using multiple partially matching references. The authors claim that the tool has a good repair rate and generates quality feedback. However, there needs to be an extensive evaluation of its usability in other programming languages. FixML [56] uses statistical error localization and search-based program synthesis to generate feedback for logical errors in functional programming assignments. The input to FixML includes an incorrect OCaml program, a test case, and an instructor-generated solution. This approach was able to fix 43% of the 497 student submissions and provide corrective suggestions as feedback. **Fast and accurate program repair (FAPR)** [62] employs a high-speed token-level comparison to locate the code that matches the incorrect submission. Program encoding is used to retrieve more accurate candidate references. Based on the identified set of candidate references, the programs with the minimal statement level difference are chosen and normalized. The normalized code is validated using testing to obtain an accurate fix. Unlike the other methods, this approach is cross-language portable (supports C++ and C) and has a high repair rate. Edmison and Edwards [24] used Gzoltar [11], a FL library, and heat map visualization to provide feedback on CS1 programming assignments. While this approach was successful in localizing bugs at the block level, it could not localize the bug at the statement level due to ties in the suspicion value among the program statements. Additionally, students found the heatmaps difficult to understand.

4.4 LLMs

Most recently, several LLMs have been used to generate and process source code for a variety of programming tasks. Sobania et al. [97] studied the bug-fixing performance of ChatGPT on the QuixBugs [60] benchmark and found that it outperformed the standard APR approaches and achieved similar performance to Codex [13] and other deep learning approaches. In another study conducted by Zhang et al. [116], a new APR system called **Multi-modal automated repair system (MMAPR)** based on Codex was created to fix syntax and semantic errors in Python programming assignments. Results indicate that MMAPR was able to outperform the state-of-the-art syntax and semantic repair engines. In order to avoid the generation of the same incorrect patches repeatedly, Xia and Zhang [112] developed a new paradigm called conversational APR, where the model is

fed with validation feedback (test case results) along with the generated code. They observed that these dialogue-based approaches have improved the performance of the model. PyFiXV [86] is a new approach to generating high-precision feedback for syntax errors in Python programming assignments. In this approach, which is based on Codex, feedback that includes the correct program and an explanation of the errors is generated in the first phase. PyFiXV introduced a novel runtime validation mechanism to ensure the suitability of the generated feedback in the subsequent phase. The findings of this study demonstrate its effectiveness in generating quality feedback. Kogi [50] a learning support system that integrates ChatGPT into a Jupyter environment generated timely advice for students. The study demonstrated a significant reduction in unresolved student errors in two courses: Algorithms and Data Structures. Balse et al. [6] evaluate GPT-3's potential in providing feedback for Python programming assignments. The model demonstrated variable accuracy ranging from 32% to 93%, and it often generated incorrect responses. These responses indicate GPT-3's limitations for reliable use in programming education. Another study by Balse et al. [5] examined GPT-3.5-turbo's explanation quality for Python programming assignments and rated it as comparable to peer-generated ones. Although 50% of the LLM explanation included inaccuracies, 93% correctly identified the logical error in the code. These findings also indicate that in large CS1 classes, **teaching assistants (TAs)** could effectively use the LLMs to grade the submissions.

Leinonen et al. [57] examine whether Codex (code-davinci-002) can enhance Python **programming error messages (PEMs)**. Although Codex was able to generate an explanation for 84% of the code, only half of these were considered correct. Furthermore, only 47% of code fix suggestions were accurate. The findings indicate that LLMs are not yet ready for production use in CS1 classes. CodingSteps [47] explored the effect of AI code generators in an introductory programming setting. A controlled experiment was conducted using a web application that uses OpenAI Codex's code-davinci-002 model to generate Python code. It was observed that novice programmers' code-authoring performance significantly improved with the help of AI generators, while their manual code-modification abilities remained unaffected. Hellas et al. [38] examined the effectiveness of LLMs to detect issues in Dart code. The findings indicate that GPT-3.5 outperformed Codex in most cases, although neither model detected all the issues. The study highlights the unreliability of the results and suggests the need for future customization. In another study, Koutchme et al. [49] explore the effectiveness of open LLMs for program repair to assist novice programmers. The study evaluated the performance of Incoder [28] model for code infilling and experimented with four programming datasets and reported satisfactory performance. They concluded that, with appropriate pre-training strategies and more sophisticated algorithms, LLMs in the future will be able to address the various debugging challenges. Another study by Santos et al. [96] examined the role of code context in enhancing PEMs using erroneous Java code from the Blackbox Mini dataset [10]. The findings confirm that code context improves the accuracy of GPT-4, demonstrating its utility for novice programmers. CodeHelp [59] is an LLM-based tool designed to support programming students without offering complete solutions. A sufficiency check is carried out to evaluate the accuracy of the prompt, and refinements are made to generate precise results. Although the findings are positive, the paper advocates for comprehensive evaluations using diverse metrics. GPT-3.5 [83] was used to provide personalized feedback for C# learners in an automated assessment platform. Findings revealed that 46% of students rated the hints as useful. The study also highlighted the issues related to overreliance of AI generated feedback.

Dcc-help [101] presents an integration of the OpenAI ChatGPT 3.5 API into the Debugging C Compiler's compile and runtime processes (Dcc-help) to provide contextual PEMs for novice programmers. The findings indicate that students expressed greater satisfaction with the feedback generated by the Dcc-help compared to the cryptic error messages from compilers. Ouh et al. [79] examine the effectiveness of ChatGPT in generating solutions for undergraduate Java programming

assignments. The model was able to produce accurate results; however, it struggled with exercises involving non-textual elements. Xia et al. [111] conducted a comprehensive evaluation of nine state-of-the-art LLMs and observed that they outperformed current APR approaches. The researchers also suggested that the performance could be further improved by increasing the sample size and integrating a fix template.

Overall analysis of these studies indicates that LLMs are being used for multiple use cases such as code generation, enhancing error messages, code fix recommendations, generating assignments, and repairing code. The findings indicate that they have outperformed the various automatic program repair techniques in many tasks. The experiments were mostly carried out using Codex, InCoder, GPT-3, GPT-3.5, ChatGPT, and GPT-4 models. A comparison of performance of these models cannot be carried out as these studies relied on different datasets. However, it was observed that the GPT-3.5 model outperformed Codex when applied on the same dataset as outlined in [38]. Computing education researchers rated the feedback clarity of GPT-4 model at 96%. The findings from four papers involving ChatGPT were also positive. Students found the LLM-generated responses generally useful and helpful in completing the programming assignments. It was also observed that few-shot learning enhances the ability of LLMs to repair code, as outlined in [116].

5 Discussion

Table 3 presents a summary of the reviewed papers in chronological order, highlighting supported programming languages, feedback generation techniques, feedback types, datasets, and the key findings of each study.

5.1 RQ1. Which Computational Techniques Are Used to Generate Feedback for Programming Exercises?

Data-driven techniques, program repair-based techniques, machine learning and deep learning-based techniques, and LLMs are the widely used techniques for automated feedback generation.

Data-driven techniques use existing student data to generate feedback and thus do not require extensive efforts from the instructor in comparison to the model-tracing approaches used in the past. Clara [31] performs well on small programs, whereas SARFGEN [107] outperforms it for large, complex programs. This improvement is due to SARFGEN's use of novel program embedding technique that allows it to scale well for large programs. Refactory [41] achieves substantially higher repair rate than Clara [31] because of the transformations that it applies to the reference programs. Verifix [1], on the other hand, can generate feedback even if there is at least one sample reference solution, unlike other methods that require a lot of correct submissions. We observed that very few techniques discussed above have made their dataset and tools publicly available, which limits further evaluation of the performance of these tools. Moreover, some of the techniques are designed for a specific programming language and are therefore not language agnostic. Many of the data-driven methods were successful in generating next-step hints, which assist learners in transforming the code from its current state to an improved version. Studies suggest that incorporating visualization aids and textual explanations, along with next-step hints will enhance the learning experience. It was also evident that novice programmers preferred textual explanations alongside the next-step hints [67]. Additionally, it was observed that transformation hints are more beneficial to learners than bottom-out hints [89]. We observed that, since these systems depend on a pool of correct student submissions, it is essential to verify their accuracy.

Machine learning systems, particularly those based on neural networks [2, 3, 32, 33], are used to generate repair suggestions for compilation errors in multiple programming languages. To enhance the pedagogical value of the response, TEGCER [3] generated similar worked out examples as feedback instead of providing direct fixes. However, this approach will not be effective for error

types that are not stored in its database. We observed that most of these systems focused on syntax errors rather than other kinds of errors. Only a limited number of systems support feedback generation for multiple programming languages, highlighting the need to design systems that are language agnostic. Additionally, such systems require large volumes of data and computing resources for training. Verifying the correctness of the data is both challenging and essential, as it influence the reliability and quality of the response.

It was observed that program synthesis-based repair is used to generate feedback for semantic and logical errors. Although APR tools have been successful in resolving errors in large codebases, the reparability was low when state-of-the-art tools (Astor [64], Nopol [19]) were specifically evaluated on programming assignments. This low repair rate is due to the existence of numerous errors in student submissions and the complex nature of the required fixes. Program repair tools currently can only repair a limited range of bugs, might generate incorrect patches, and involve high computational time. Moreover, generating a repair for a multi-location bug is again a complex task, as described in [114]. These factors adversely affect the acceptance of APR tools in CS1. Although some techniques offer a repaired program as feedback, this will not facilitate the learning process. Therefore, there is a need to process the generated repair to create useful feedback. Moreover, if the repaired program significantly deviates from the original submission, it will not provide any valuable assistance to the learner. Research studies have observed that students prefer to have smaller patches that resemble the original submission coupled with hints that help them complete the partial solution [46].

To apply automatic program repair in an educational environment, the efficacy of the existing tools must be evaluated based on student submissions. One of the popular benchmarks used to evaluate the techniques for C programming is Prutor [17]. However, there are only a limited number of publicly available benchmarks related to Java programming assignments such as Blackbox [10], IntroclassJava [23], Quixbugs [60], and Propedia [82]. In order to conduct a comprehensive evaluation of the newly proposed approaches, it is essential to make the benchmarks publicly available. Some research papers used spectrum-based FL techniques to generate feedback for programming assignments. These techniques assign a suspicion value to the program elements and are successful at localizing errors accurately at the method level. However, because of the ties that exist at the program statement level, it is difficult to precisely localize the bug.

An analysis of the papers published between 2017 and 2023 indicates that there is a shift toward adopting new AI-based techniques for feedback generation. Since 2022, we observed a notable shift toward adopting LLMs in programming education (Figure 2). Several LLMs (Codex [13], Copilot [29], Amazon's CodeWhisperer [4], and ChatGPT [78]) showcased their ability to automatically create programming assessments, provide high-quality explanations for code, and even generate code for a specific programming task. ChatGPT achieved impressive results in comparison to the standard deep learning-based APR tool; however, the overhead associated with verifying the correctness of these repair suggestions cannot be ignored. Python has become the most widely used language. While some studies have focused on block-based languages, text-based programming languages dominate the research landscape as shown in Figure 3. We observed that LLMs have outperformed the various other techniques and addressed their limitations. However it was observed that challenges such as incorrect responses and hallucination continue to hinder their widespread adoption.

Since the datasets used to evaluate the various computational techniques are not published in most cases, conducting a comparative analysis is impractical. We observed frequent reuse of the dataset ITSP [17] in multiple studies. Quixbugs-based experiments are included in this study as it includes errors that are commonly found in student submissions. While most of the feedback generation techniques targeted semantic errors in the code, some studies based on APR techniques focused on syntax errors. Figure 4 provides an overview of the various error categories addressed

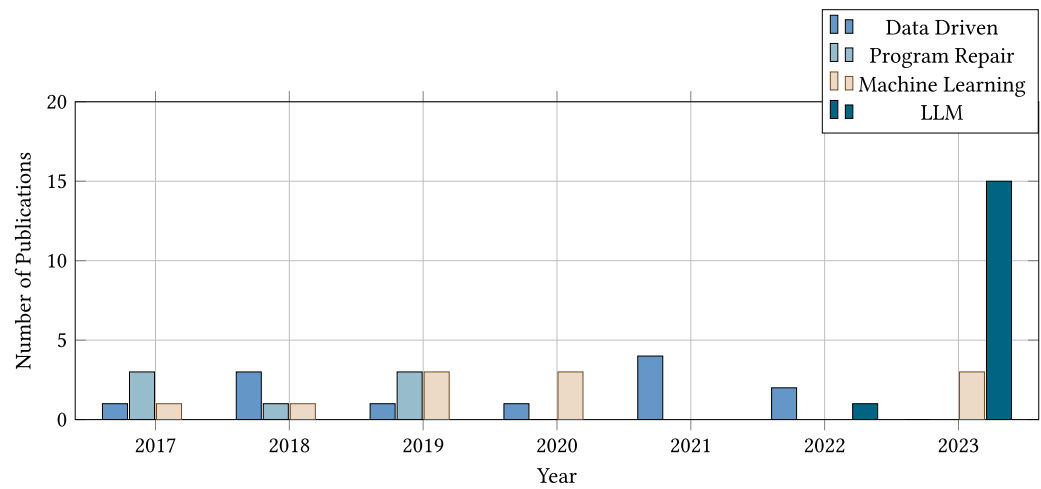


Fig. 2. Feedback generation techniques organized by year.

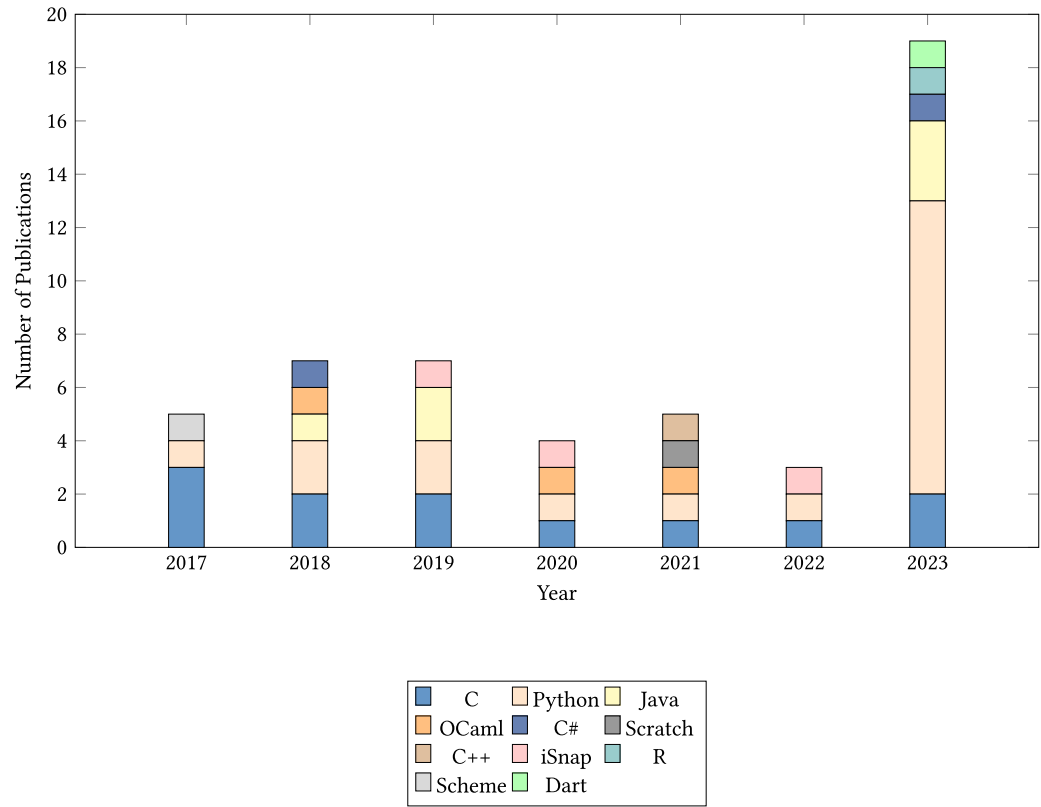


Fig. 3. Yearly count of publications for each programming language.

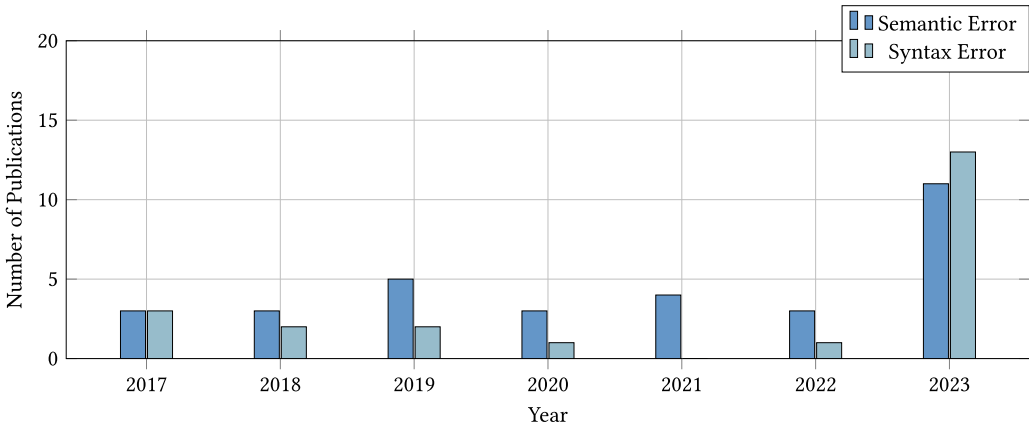


Fig. 4. Error types addressed by the various feedback generation approaches.

by the feedback generation techniques over the years. LLMs can address both types of errors, giving them an advantage over the other approaches outlined in Table 3. Moreover, building pedagogical systems using LLMs is significantly easier than automatic program repair-based techniques [59].

5.2 RQ2. What Types of Feedback Are Generated, and How Useful Are They for Learners?

Several research studies in the past have reported that most of the existing tools provide KM type feedback. The findings of this study aligns with earlier research, as we also observed numerous instances of KM-type feedback. However, newer techniques have integrated textual hints to enhance feedback quality, as shown in Table 3. Many of these techniques generate KH-type hints, which can take various forms. The main categories are:

- Next-step hints
- Hints related to bug fixing
- Example code
- Solution code/repared programs
- Code explanation

Notably, many of the LLM-based techniques in Table 3 generate KH-type feedback. As code LLMs are capable of generating customized feedback using prompt engineering and support a wide range of programming languages, they are being increasingly adopted to generate diverse forms of feedback.

In terms of the usefulness of the generated feedback, the findings column in Table 3 primarily reflects student evaluations; in certain studies, other evaluators (R: Researchers, T: Tutors, TAs) are also involved. Learners generally found the generated feedback valuable. It was also observed that providing a repaired program as feedback might not facilitate the learning process (RQ2). Research studies also indicate that in certain cases, the student might debug the code using the next-step hint without understanding the concepts. Therefore, multiple papers have emphasized the need to complement next-step hints with textual explanations, visualizations, and worked examples to enhance feedback [108].

Table 2. Summary of Previous Reviews on Automated Feedback Generation

Review	Years Covered	Context/Goal of the Study
Keuning et al. [48]	Up to 2015	Examined the various tools and the associated feedback types, computational techniques, adaptability, and evaluation methods.
Cavalcanti et al. [12]	2009–2018	Evaluated the impact of automated feedback on learner’s performance and instructor workload.
McBroom et al. [68]	2014–2018	Surveyed the various techniques in the context of HINTS.
Paiva et al. [81]	2010–first half of 2021	Examined the exercises supported, security of code execution, testing techniques, feedback types, and the pedagogical insights.
Messer et al. [73]	2017–2021	Categorized the papers based on assessed skills, computational techniques, automation level, and evaluation method.

5.3 Comparison with Related Reviews

In this section, we compare our results with previous reviews to highlight similarities and differences in our findings, with Table 2 providing an overview of those reviews.

Keuning et al. [48] focused on papers published until 2015. The study highlighted that most of the tools focused on identifying mistakes rather than providing guidance on how to proceed (KH). They observed that the feedback tools exhibit insufficient customization options, and extensive evaluations are needed to verify the quality of the feedback. Their study found that most of the tools provided KM-type feedback, which aligns with our observation. However, our findings indicate an increasing trend in KH-type feedback generation in the recent years. We also observed instances of KCR-type responses, often in the form of repaired programs in certain computational techniques. However, KCR-type feedback limits learners’ engagement in problem-solving and critical thinking. Cavalcanti et al. [12] identified similar computational techniques to those discussed in our study. However, our study covers computational techniques based on program repair and LLMs that were not included in their work. We also observed that several new techniques share foundational elements similar to those explained in the review by McBroom et al. [68]. Although they reviewed some of the automatic program repair techniques, our study covers a larger number of papers. Paiva et al. [81] surveyed automated assessment in CS covering exercise formats, security and testing, feedback types, and instructional value. According to the authors, KH-type feedback includes meaningful guidance for correcting bugs, providing a hint for the next step when a learner is lost, and suggesting enhancements to solutions irrespective of their correctness. We found similar categories of KH-type feedback as those outlined in this study at various scaffolding levels. Messer et al. [73] observed that most of the tools provide limited feedback such as KR (correct, incorrect) as they rely on automated testing. This can be attributed to the study’s focus on AAT systems. While our RQs overlap with those in [73], they did not examine the error types addressed by the feedback generation techniques or the associated feedback type. Moreover, there was limited exploration of the various techniques that use LLMs. In comparison to these existing studies, our findings point to a growing emphasis toward richer, more informative feedback instead of KR-type feedback. Our study presents an overview of computational techniques, error types, supported programming languages, datasets used, feedback classification, and evaluation outcomes.

Table 3. Overview of Automated Feedback Generation Techniques

Study	Year	Languages	DD	ML	PS	LLM	SE	SyE	Feedback Type	Dataset Description	Findings
ITAP [90]	2017	Python	●	○	○	○	●	○	Next-step hint	Locally collected student solutions	Next-step hints helpful in 75% of cases, solution hints in 86% of cases.
GradeIT [84]	2017	C	○	○	●	○	○	●	Enhanced compiler error message	Locally collected student solutions (Prutor [17])	Average feedback rating 2.4/3.
ITSP [115]	2017	C	○	○	○	○	●	○	Fully and partially repaired program	Locally collected student solutions (Prutor [17])	Unsure about how to use hints.
Phothilimthana and Sridhara [85]	2017	Scheme	○	○	●	○	●	●	Hint	Locally collected student solutions	Mix of both positive and negative responses.
DeepFix [33]	2017	C	○	●	○	○	○	●	Error location, correction statement	Locally collected student solutions	27% programs were completely and 19% programs partially fixed (R).
Clara [31]	2018	C, Python	●	○	○	○	●	○	Error location, correction statement	External data: MITx MOOC	Feedback rated 3.4/5.
TRACER [2]	2018	C (can be ported to C++, Java)	○	●	○	○	○	●	Error location, correction statement/repaired program	Locally collected student solutions	79.27% accurate repair for single line error and 44% for multiple line edit as observed (R).
SARFGEN [107]	2018	C#, Python	●	○	○	○	●	○	Error location, correction statement, repaired program	External data: Microsoft DEV204.1x edX course, Microsoft Code-Hunt platform	Concise and useful feedback 89.7%, user rating on a 1–5 scale is 3.74.
FixML [56]	2018	OCaml	○	○	●	○	●	○	Error location, correction statement	Locally collected student solutions	Usefulness in learning OCaml (72%), helps to understand the mistakes (50%).
Sensibility [95]	2018	Java	●	○	○	○	○	●	Error location, correction statement	External data: Java code from GitHub	Mean reciprocal rank 0.46 (positive).
RLAssist [32]	2019	C	○	●	○	○	○	●	Error location, correction statement	Locally collected student solutions	Fixes 24.4% programs completely and 18.8% programs partially (R).
IPT [58]	2019	Python	○	○	●	○	●	○	Fix errors, identify error type	Locally collected student solutions	Fixed 88.75% of programs (R).
Refactory [41]	2019	Python	○	○	●	○	●	○	Error location, correction statement	Locally collected student solutions	Repair rate 71.65%, higher than baseline model (R).
iSnap [67]	2019	Snap	●	○	○	○	●	○	Next-step hints	Locally collected student solutions	Textual explanations with code preferred by learners.
Gzoltar [24]	2019	Java	○	○	●	○	●	○	Error location	Locally collected student solutions (plug-in for Web-CAT)	Heat maps are hard to use and did not improve students' class performance.
Annette [18]	2019	Java	○	●	○	○	●	○	Problem-solving feedback	Locally collected student solutions	Helpful or slightly helpful (42%), feedback did not match the situation in the code (41%) and did not understand the feedback (17%).

(Continued)

Table 3. Continued

Study	Year	Languages	DD	ML	PS	LLM	SE	SyE	Feedback Type	Dataset Description	Findings
TEGGER [3]	2019	C	o	●	o	o	o	●	Example of fixes for similar error	Locally collected student solutions (Prutor [17])	Resolved errors 25% faster than human tutors.
StepTutor [108]	2020	Snap	●	o	o	o	●	o	Example code to show the next step	Locally collected student solutions	Next-step hints useful.
MACER [14]	2020	C	o	●	o	o	o	●	Repaired program	Locally collected student solutions	20% superior to other approaches at suggesting fixes (R).
PABLO [16]	2020	Python	o	●	o	o	●	o	Error location, correction state-ment/hints	External data: PythonTutor	FL 59–77%, useful information 36% (R).
Rite [94]	2020	Ocaml	o	●	o	o	●	o	Error location, Re-paired program	Locally collected student solutions	High-quality repairs (R).
Codechecker [89]	2021	Python	●	o	o	o	●	o	Next-step hints	Locally collected student solutions	2x higher accuracy than baselines and 8% more than the other sophisticated learning algorithms (R).
Cafe [98]	2021	Ocaml	●	o	o	o	●	o	Repaired program	Locally collected student solutions	83% of incorrect submissions were fixed (R) and generated feedback received scores 4 or 5 out of 5.
FAPR [62]	2021	C, C++	●	o	o	o	●	o	Repaired program	Locally collected student solutions and external data: OpenJudge	Repairs for over 95.5% of incorrect solutions. 66.4% of repairs helpful.
CATNIP [77]	2021	Scratch	●	o	o	o	●	o	Next-step hints	Locally collected student solutions	Generated hints for functional improvements, number of tests passed increased from an average of 12.36–16.64 (R).
Verifix [1]	2022	C	●	o	o	o	●	o	Repaired program	Locally collected student solutions (Prutor [17])	Repair Quality and Feedback—mean/median value > = 3.8 (T).
AIF [65]	2022	iSnap	●	o	o	o	●	o	corrective feedback	Locally collected student solutions	Corrective feedback helped students complete task faster.
MMAPR [116]	2022	Python	o	o	o	●	●	●	Repaired program	Locally collected student solutions	Codex/Overall repair rate without few shots (peers' solutions as few shots) was 86.71%, and with solutions achieved 100% in comparison to baseline ARP systems (R).
Kogi [50]	2023	Python	o	o	o	●	●	●	Bug fix suggestions	Student help requests	ChatGPT/Positive feedback, unresolved errors addressed faster.
Balse et al. [6]	2023	Python	o	o	o	●	●	●	Check, critique, suggest changes	Locally collected student solutions	GPT-3/Code correctness 71.13%, critique 62.63%, and useful suggestions 61.95% (R).
Balse et al. [5]	2023	Python	o	o	o	●	o	●	Code explanations	Locally collected student solutions	GPT-3.5/Incorrect statement (50%), LLM detects logical error (93%) (TAs).
Leinonen et al. [57]	2023	Python	o	o	o	●	o	●	Code fix suggestions	Python error messages	Error messages comprehensible 88%, accurate 57% (R).
Pankiewicz and Baker [83]	2023	C#	o	o	o	●	●	●	Code fix suggestions	Locally collected student solutions	GPT-3.5/46% positive about GPT-generated hints, 19% suggested that it is not useful.

(Continued)

Table 3. Continued

Study	Year	Languages	DD	ML	PS	LLM	SE	SyE	Feedback Type	Dataset Description	Findings
PyFixV [86]	2023	Python	○	○	○	●	○	●	High precision feedback	External data: Tigerlython, Codeforces	Codex/Generated high-precision feedback (R).
rDSA/Uts [106]	2023	R	○	●	○	○	●	○	Suggestions for corrections	Locally collected student solutions	General usefulness, clarity of the feedback for the completely wrong commands, and exercise-solving usefulness received positive response, however, negative feedback for partially incorrect commands' clarity.
Capex [34]	2023	C	○	●	○	○	○	●	Repaired program	Locally collected student solutions (Prutor [17])	4–5% higher repair accuracy (R).
PyPer [34]	2023	Python	○	●	○	○	○	●	Repaired program	Locally collected student solutions (Prutor [17])	Generalizes well on unseen data and high repair rate (R).
CodingSteps [47]	2023	Python	○	○	○	●	●	●	Python code	Locally collected student solutions	Codex/1.15X increased completion rate and 1.8X higher scores.
Hellas et al. [38]	2023	Dart	○	○	○	●	●	●	Detect bugs	Locally collected student solutions	GPT-3.5 outperformed Codex in issue detection (R).
Koutchene et al. [49]	2023	Python	○	○	○	●	●	○	Code fix suggestions	Locally collected student solutions (Dublin City University data University of New Caledonia data, National University of Singapore data) and external data: Quixbugs [60]	InCoder/Repaired 13 out of 28 programs (R).
Santos et al. [96]	2023	Java	○	○	○	●	○	●	Repaired program	Blackbox Mimi [10]	GPT-4/Computing education researchers' ratings: feedback clarity 96%, correct explanation 59% (T).
Sobania et al. [97]	2023	Python	○	○	○	●	●	○	Code explanations	External data: Quixbugs [60]	ChatGPT/Bug fixes for 19 projects, additional hint fix for 31/40 projects (R).
CodeHelp [59]	2023	Python	○	○	○	●	●	●	Enhanced Error messages	Student help request	Assist in finishing task (80%), helps in learning new concepts (63%).
Dcc-help [101]	2023	C	○	○	○	●	●	●	Code fix suggestions	Student help request	LLM explanations more accurate for compile-time errors (90%) than runtime errors (75%). This could be due to the lack of runtime error context for the LLM (R).
Ouh et al. [79]	2023	Java	○	○	○	●	○	○	Repaired program	Java programming exercises	ChatGPT/Positive comments mostly, however, difficulty with non-textual descriptions or class files (R).
Xia and Zhang [112]	2023	Python, Java	○	○	○	●	●	○	Repaired program	External data: Quixbugs [60]	CODEGEN-MONO/CODEGEN-MULTI/Codex/ChatGPT/Higher number of correct C/plausible patches compared to baseline models (R).

DD, data driven; ML, machine learning; PS, program synthesis (RQ1); SE, semantic/logical error; SyE, syntax error; Findings, columns primarily reflect student evaluations; row details indicate other evaluators (R: Researchers, T: Tutors) (RQ2).

5.4 Threats to Validity

Our study examines the literature published between 2017 and 2023, but it should be noted that further studies have been published since this review was written, and the field continues to develop rapidly. Restricting the search to ACM DL, IEEE Xplore, and Google Scholar might have resulted in some studies being missed. We excluded a large number of papers based on a review of the title and abstract; the research was conducted by the first author under the supervision of co-authors, and therefore, there is a possibility of relevant papers being missed or mislabeling of relevant papers. We opted not to do snowball search, as the initial search provided a large and relevant set of studies; however, it is possible that some studies were missed. We excluded certain recent automatic program repair tools as the evaluation was not carried out on student submissions. As new code LLMs continue to be released, it is likely that performance improvements beyond those outlined in this article will be observed.

6 Conclusions and Future Work

In this survey, we presented the recent developments in automatic feedback generation for programming exercises. We found that various computational techniques based on machine learning, data-driven techniques, automated program repair, and LLMs are used for feedback generation (RQ1). These methods generate various forms of feedback, such as providing next-step hints with textual explanations, identifying faulty lines, and producing repaired programs as output. Most studies point out the overall benefit of different types of hints, noting a trend toward providing KH-type feedback and suggesting potential areas for improvements (RQ2). However, there are several issues that need to be addressed to achieve widespread acceptance of these methods in an educational setting. Based on this survey, we present some recommendations for future work:

- Programming language-specific techniques are evaluated on benchmarks that are not publicly available. As a result, a comparison becomes impractical. This highlights the need for fully anonymized student submissions for future research purposes. Another strategy could be leveraging LLMs to generate these datasets.
- Most of the automatic program repair tools require complicated installation steps and are not very novice-friendly. Therefore, it is essential to develop web-based systems or IDE plugins that students and teachers can easily use. Additionally, program repair techniques have limitations in repairing all kinds of bugs (semantic and syntax), and researchers should solve this problem to enhance their adoption in a pedagogical environment. The results from LLMs in this regard appear very promising. It would be useful to consider integrating LLM-based pedagogical systems into LMS rather than deploying them as standalone tools.
- Although researchers have developed numerous techniques, most of them are tailored to specific programming languages and therefore lack cross-language portability. Therefore, there is a need to formulate approaches that are language agnostic. Code LLMs could be beneficial in this context as they are trained on large code bases of multiple programming languages. However, there have so far been no studies of this, and it is likely that the performance of an LLM within this context will be linked to the amount of language-specific code in its training data.
- The objective of feedback is to highlight underlying errors so that students can correct similar errors themselves in the future. More studies are needed to identify what information should be part of the feedback within different programming environments (visual programming vs. standard programming languages) and how to evaluate the quality of the feedback.
- Most research studies on automated feedback generation targets CS1 and CS2, highlighting the need to explore its role in more advanced courses. For example, there is a need for more specialized approaches within the kind of domain-specific language subsets used within

disciplines such as machine learning and image processing, especially since these courses are increasingly at the forefront of CS education.

- Even though LLMs have significantly outperformed alternative approaches, additional studies are required to identify the potential benefits and challenges associated with them. As LLM responses often include functional code, adding guardrails is important, as offering code along with feedback does not contribute to learning. Addressing the challenges related to LLMs such as hallucinations and incorrect responses is essential as precision is critical in a pedagogical setting. The LLMs used in most of the studies were proprietary and lacked transparency, customization options, and cost-effectiveness for educational institutions. There is a need for more research on the use of open LLMs.

As AI and LLMs continue to advance, we foresee significant improvements in the field of automated feedback generation systems that will enrich the student experience.

References

- [1] Umair Z. Ahmed, Zhiyu Fan, Jooyong Yi, Omar I. Al-Bataineh, and Abhik Roychoudhury. 2022. Verifix: Verified repair of programming assignments. *ACM Transactions on Software Engineering and Methodology* 31, 4 (2022), 1–31. DOI: <https://doi.org/10.1145/3510418>
- [2] Umair Z. Ahmed, Pawan Kumar, Amey Karkare, Purushottam Kar, and Sumit Gulwani. 2018. Compilation error repair: For the student programs, from the student programs. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET '18)*. ACM, New York, NY, 78–87. DOI: <https://doi.org/10.1145/3183377.3183383>
- [3] Umair Z. Ahmed, Renuka Sindhgatta, Nisheeth Srivastava, and Amey Karkare. 2019. Targeted example generation for compilation errors. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, San Diego, CA, 327–338. DOI: <https://doi.org/10.1109/ASE.2019.00039>
- [4] Desai Ankur and Deo Atul. 2022. Amazon CodeWhisperer. Retrieved August 21, 2023 from <https://aws.amazon.com/blogs/machine-learning/introducing-amazon-codewhisperer-the-ml-powered-coding-companion/>
- [5] Rishabh Balse, Viraj Kumar, Prajish Prasad, and Jayakrishnan Madathil Warriem. 2023. Evaluating the quality of LLM-generated explanations for logical errors in CS1 student programs. In *Proceedings of the 16th Annual ACM India Compute Conference (COMPUTE '23)*. ACM, New York, NY, 49–54. DOI: <https://doi.org/10.1145/3627217.3627233>
- [6] Rishabh Balse, Bharath Valaboju, Shreya Singhal, Jayakrishnan Madathil Warriem, and Prajish Prasad. 2023. Investigating the potential of GPT-3 in providing feedback for programming assessments. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE '23)*. ACM, New York, NY, 292–298. DOI: <https://doi.org/10.1145/3587102.3588852>
- [7] Brett A. Becker, Paul Denny, Raymond Pettit, Durell Bouchard, Dennis J. Bouvier, Brian Harrington, Amir Kamil, Amey Karkare, Chris McDonald, Peter-Michael Osera, et al. 2019. Compiler error messages considered unhelpful: The landscape of text-based programming error message research. In *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '19)*. ACM, New York, NY, 177–210. DOI: <https://doi.org/10.1145/3344429.3372508>
- [8] Brett A. Becker and Keith Quille. 2019. 50 years of CS1 at SIGCSE: A review of the evolution of introductory programming education research. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19)*. ACM, New York, NY, 338–344. DOI: <https://doi.org/10.1145/3287324.3287432>
- [9] Jens Bennedsen and Michael E. Caspersen. 2019. Failure rates in introductory programming: 12 years later. *ACM Inroads* 10, 2 (2019), 30–36. DOI: <https://doi.org/10.1145/3324888>
- [10] Neil Christopher, Charles Brown, Michael Kölling, Davin McCall, and Ian Utting. 2014. Blackbox: A large scale repository of novice programmers' activity. In *Proceedings of the 45th ACM Technical Symposium on Computer Science Education (SIGCSE '14)*. ACM, New York, NY, 223–228. DOI: <https://doi.org/10.1145/2538862.2538924>
- [11] José Campos, André Ribeiro, Alexandre Perez, and Rui Abreu. 2012. GZoltar: An eclipse plug-in for testing and debugging. In *Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering (ASE '12)*. ACM, New York, NY, 378–381. DOI: <https://doi.org/10.1145/2351676.2351752>
- [12] Anderson Pinheiro Cavalcanti, Arthur Barbosa, Ruan Carvalho, Fred Freitas, Yi-Shan Tsai, Dragan Gašević, and Rafael Ferreira Mello. 2021. Automatic feedback in online learning environments: A systematic literature review. *Computers and Education: Artificial Intelligence* 2 (2021), 100027. DOI: <https://doi.org/10.1016/j.caeai.2021.100027>
- [13] Mark Chen, Jerry Twarek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>

- [14] Darshak Chhatbar, Umair Z. Ahmed, and Purushottam Kar. 2020. MACER: A modular framework for accelerated compilation error repair. In *Proceedings of the 21st International Conference on Artificial Intelligence in Education (AIED '20)*. Springer-Verlag, Berlin, 106–117. DOI : https://doi.org/10.1007/978-3-030-52237-7_9
- [15] Albert T. Corbett, John R. Anderson, and Eric G. Patterson. 1990. Student modeling and tutoring flexibility in the Lisp Intelligent Tutoring System. In *Intelligent Tutoring Systems: At the Crossroads of Artificial Intelligence and Education*. C. Frasson and G. Gauthier (Eds.), Ablex Norwood, NJ, 83–106.
- [16] Benjamin Cosman, Madeline Endres, Georgios Sakkas, Leon Medvinsky, Yao-Yuan Yang, Ranjit Jhala, Kamalika Chaudhuri, and Westley Weimer. 2020. PABLO: Helping novices debug Python code through data-driven fault localization. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education (SIGCSE '20)*. ACM, New York, NY, 1047–1053. DOI : <https://doi.org/10.1145/3328778.3366860>
- [17] Rajdeep Das, Umair Z. Ahmed, Amey Karkare, and Sumit Gulwani. 2016. Prutor: A system for tutoring CS1 and collecting student programs for analysis. arXiv:1608.03828. Retrieved from <https://doi.org/10.48550/arXiv.1608.03828>
- [18] Melissa Day, Manohara Rao Penumala, and Javier Gonzalez-Sanchez. 2019. Annete: An intelligent tutoring companion embedded into the eclipse IDE. In *Proceedings of the 2019 IEEE 1st International Conference on Cognitive Machine Intelligence (CogMI)*. IEEE, Los Angeles, CA, 71–80. DOI : <https://doi.org/10.1109/CogMI48466.2019.00018>
- [19] Favio DeMarco, Jifeng Xuan, Daniel Le Berre, and Martin Monperrus. 2014. Automatic repair of buggy if conditions and missing preconditions with SMT. In *Proceedings of the 6th International Workshop on Constraints in Software Testing, Verification, and Analysis (CSTVA 2014)*. ACM, New York, NY, 30–39. DOI : <https://doi.org/10.1145/2593735.2593740>
- [20] Paul Denny, Brett A. Becker, Michelle Craig, Greg Wilson, and Piotr Banaszkiewicz. 2019. Research this! Questions that computing educators most want computing education researchers to answer. In *Proceedings of the 2019 ACM Conference on International Computing Education Research (ICER '19)*. ACM, New York, NY, 259–267. DOI : <https://doi.org/10.1145/3291279.3339402>
- [21] Paul Denny, Andrew Luxton-Reilly, and Ewan Tempero. 2012. All syntax errors are not equal. In *Proceedings of the 17th ACM Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE '12)*. ACM, New York, NY, 75–80. DOI : <https://doi.org/10.1145/2325296.2325318>
- [22] Christopher Douce, David Livingstone, and James Orwell. 2005. Automatic test-based assessment of programming: A review. *Journal on Educational Resources in Computing* 5, 3 (2005), 4–es. DOI : <https://doi.org/10.1145/1163405.1163409>
- [23] Thomas Durieux and Martin Monperrus. 2016. IntroClassJava: A benchmark of 297 small and buggy Java programs. Retrieved from <https://hal.archives-ouvertes.fr/hal-01272126>
- [24] Bob Edmison and Stephen H. Edwards. 2019. Experiences using heat maps to help students find their bugs: Problems and solutions. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19)*. ACM, New York, NY, 260–266. DOI : <https://doi.org/10.1145/3287324.3287474>
- [25] Stephen H. Edwards. 2021. Automated feedback, the next generation: Designing learning experiences. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education (SIGCSE '21)*. ACM, New York, NY, 610–611. DOI : <https://doi.org/10.1145/3408877.3437225>
- [26] Stephen H. Edwards and Manuel A. Perez-Quinones. 2008. Web-CAT: Automatically grading programming assignments. In *Proceedings of the 13th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE '08)*. ACM, New York, NY, 328. DOI : <https://doi.org/10.1145/1384271.1384371>
- [27] Andrew Ettles, Andrew Luxton-Reilly, and Paul Denny. 2018. Common logic errors made by novice programmers. In *Proceedings of the 20th Australasian Computing Education Conference (ACE '18)*. ACM, New York, NY, 83–89. DOI : <https://doi.org/10.1145/3160489.3160493>
- [28] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen Tau Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A generative model for code infilling and synthesis. arXiv:2204.05999. Retrieved from <https://arxiv.org/abs/2204.05999>
- [29] Nat Friedman. 2021. Introducing GitHub Copilot: Your AI Pair Programmer. GitHub. Retrieved from <https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/>
- [30] Ken Goldman, Paul Gross, Cinda Heeren, Geoffrey Herman, Lisa Kaczmarczyk, Michael C. Loui, and Craig Zilles. 2008. Identifying important and difficult concepts in introductory computing courses using a delphi process. In *Proceedings of the 39th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '08)*. ACM, New York, NY, 256–260. DOI : <https://doi.org/10.1145/1352135.1352226>
- [31] Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices* 53, 4 (2018), 465–480. DOI : <https://doi.org/10.1145/3192366.3192387>
- [32] Rahul Gupta, Aditya Kanade, and Shirish Shevade. 2019. Deep reinforcement learning for syntactic error repair in student programs. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence and 31st Innovative Applications of Artificial Intelligence Conference and 9th AAAI Symposium on Educational Advances in Artificial Intelligence (AAAI '19/AAAI '19/EAAI '19)*. AAAI Press, Honolulu, Hawaii, Article 115, 8 pages. DOI : <https://doi.org/10.1609/aaai.v33i01.3301930>

- [33] Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish Shevade. 2017. DeepFix: Fixing common C language errors by deep learning. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence (AAAI '17)*. AAAI Press, San Francisco, California, 1345–1351.
- [34] Sharath H. Padmanabha, Fahad Shaikh, Mayank Bansal, Debanjan Chatterjee, Preeti Singh, Amey Karkare, and Purushottam Kar. 2023. Advances in automated pedagogical compile-time error repair. In *Proceedings of the 16th Innovations in Software Engineering Conference (ISEC '23)*. ACM, New York, NY, Article 11, 11 pages. DOI: <https://doi.org/10.1145/3578527.3578535>
- [35] N. R. Haddaway, M. J. Page, C. C. Pritchard, and L. A. McGuinness. 2022. PRISMA2020: An R package and shiny app for producing PRISMA 2020-compliant flow diagrams, with interactivity for optimised digital transparency and open synthesis. *Campbell Systematic Reviews* 18, e1230 (2022), 12 pages. DOI: <https://doi.org/10.1002/cl2.1230>
- [36] Georgiana Haldeman. 2021. *Automated Feedback Generation for Programming Assignments*. Ph.D. Dissertation. Rutgers The State University of New Jersey, School of Graduate Studies.
- [37] Qiang Hao, Jack P. Wilson, Camille Ottaway, Naitra Iriumi, Kai Arakawa, and David H. Smith. 2019. Investigating the essential of meaningful automated formative feedback for programming assignments. In *Proceedings of the 2019 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, Memphis, TN, 151–155. DOI: <https://doi.org/10.1109/VLHCC.2019.8818922>
- [38] Arto Hellas, Juho Leinonen, Sami Sarsa, Charles Koutchme, Lilja Kujanpää, and Juha Sorva. 2023. Exploring the responses of large language models to beginner programmers' help requests. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1 (ICER '23)*. ACM, New York, NY, 93–105. DOI: <https://doi.org/10.1145/3568813.3600139>
- [39] Michael T. Helmick. 2007. Interface-based programming assignments and automatic grading of java programs. In *Proceedings of the 12th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE '07)*. ACM, New York, NY, 63–67. DOI: <https://doi.org/10.1145/1268784.1268805>
- [40] Trong Bui Hieu and S. M. F. D. Syed Mustapha. 2018. Automated data-driven hint generation in intelligent tutoring systems for code-writing: On the road of future research. *International Journal of Emerging Technologies in Learning (iJET)* 3, 9 (2018), 174–189. DOI: <https://doi.org/10.3991/ijet.v13i09.8023>
- [41] Yang Hu, Umair Z. Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2019. Re-factoring based program repair applied to programming assignments. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, San Diego, California, 388–398. DOI: <https://doi.org/10.1109/ASE.2019.00044>
- [42] Petri Ihantola, Tuukka Ahoniemi, Ville Karavirta, and Otto Seppälä. 2010. Review of recent systems for automatic assessment of programming assignments. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research (Koli Calling '10)*. ACM, New York, NY, 86–93. DOI: <https://doi.org/10.1145/1930464.1930480>
- [43] Huzaifah Ismail and Muhammad Modi Lakulu. 2020. A critical review on recent proposed automated programming assessment tool. *Psychology and Education* 57, 8 (2020), 1049–1060.
- [44] J. Jackson, M. Cobb, and C. Carver. 2005. Identifying top Java errors for novice programmers. In *Proceedings Frontiers in Education 35th Annual Conference*. IEEE, Indianapolis, IN, T4C–24–T4C–27. DOI: <https://doi.org/10.1109/FIE.2005.1611967>
- [45] Johan Jeuring, Alex Gerdes, and Bastiaan Heeren. 2012. Ask-Elle: A Haskell Tutor. In *21st Century Learning for 21st Century Skills*. Andrew Ravenscroft, Stefanie Lindstaedt, Carlos Delgado Kloos, and Davinia Hernández-Leo (Eds.), Springer, Berlin, 453–458.
- [46] Johan Jeuring, Hieke Keuning, Samiha Marwan, Dennis Bouvier, Cruz Izu, Natalie Kiesler, Teemu Lehtinen, Dominic Lohr, Andrew Peterson, and Sami Sarsa. 2022. Towards giving timely formative feedback and hints to novice programmers. In *Proceedings of the 2022 Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '22)*. ACM, New York, NY, 95–115. DOI: <https://doi.org/10.1145/3571785.3574124>
- [47] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J. Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. ACM, New York, NY, Article 455, 23 pages. DOI: <https://doi.org/10.1145/3544548.3580919>
- [48] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education (TOCE)* 19, 1 (2018), 1–43. DOI: <https://doi.org/10.1145/3231711>
- [49] Charles Koutchme, Sami Sarsa, Juho Leinonen, Arto Hellas, and Paul Denny. 2023. Automated program repair using generative models for code infilling. In *Artificial Intelligence in Education*. Ning Wang, Genaro Rebolledo-Mendez, Noboru Matsuda, Olga C. Santos, and Vania Dimitrova (Eds.), Springer Nature Switzerland, Cham, 798–803.
- [50] Kimio Kuramitsu, Yui Obara, Miyu Sato, and Momoka Obara. 2023. Kogi: A seamless integration of ChatGPT into jupyter environments for programming education. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on SPLASH-E (SPLASH-E 2023)*. ACM, New York, NY, 50–59. DOI: <https://doi.org/10.1145/3622780.3623648>

- [51] Adidah Lajis, Shahidatul Arfah Baharudin, Diyana Ab Kadir, Nadilah Mohd Ralim, Haidawati Mohd Nasir, and Normaziah Abdul Aziz. 2018. A review of techniques in automatic programming assessment for practical skill test. *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)* 10, 2–5, (Jul. 2018), 109–113. Retrieved from <https://jtec.utm.my/jtec/article/view/4394>
- [52] Nguyen-Thinh Le, Sven Strickroth, Sebastian Gross, and Niels Pinkwart. 2013. A review of AI-supported tutoring approaches for learning programming. In *Advanced Computational Methods for Knowledge Engineering*. Ngoc Thanh Nguyen, Tien van Do, and Hoai An le Thi (Eds.), Springer International Publishing, Heidelberg, 267–279. DOI : https://doi.org/10.1007/978-3-319-00293-4_20
- [53] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2011. Genprog: A generic method for automatic software repair. *IEEE Transactions on Software Engineering* 38, 1 (2011), 54–72.
- [54] Claire Le Goues, Michael Pradel, Abhik Roychoudhury, and Satish Chandra. 2021. Automatic program repair. *IEEE Software* 38, 4 (2021), 22–27. DOI : <https://doi.org/10.1109/MS.2021.3072577>
- [55] Haden Hooyeon Lee. 2021. Effectiveness of real-time feedback and instructive hints in graduate CS courses via automated grading system. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education (SIGCSE '21)*. ACM, New York, NY, 101–107. DOI : <https://doi.org/10.1145/3408877.3432463>
- [56] Junho Lee, Dowon Song, Sunbeom So, and Hakjoo Oh. 2018. Automatic diagnosis and correction of logical errors for functional programming assignments. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018), 1–30. DOI : <https://doi.org/10.1145/3276528>
- [57] Juho Leinonen, Arto Hellas, Sami Sarsa, Brent Reeves, Paul Denny, James Prather, and Brett A. Becker. 2023. Using large language models to enhance programming error messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE '23)*. ACM, New York, NY, 563–569. DOI : <https://doi.org/10.1145/3545945.3569770>
- [58] Yu Liang, Wenjun Wu, Lisha Wu, and Meng Wang. 2019. Inferring how novice students learn to code: Integrating automated program repair with cognitive model. In *Big Data*. Hai Jin, Xuemin Lin, Xueqi Cheng, Xuanhua Shi, Nong Xiao, and Yihua Huang (Eds.), Springer Singapore, Singapore, 46–56. DOI : https://doi.org/10.1007/978-981-15-1899-7_4
- [59] Mark Liffiton, Brad E. Sheese, Jaromir Savelka, and Paul Denny. 2023. CodeHelp: Using large language models with guardrails for scalable support in programming classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. ACM, New York, NY, 1–11. DOI : <https://doi.org/10.1145/3631802.3631830>
- [60] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. QuixBugs: A multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion '17)*. ACM, New York, NY, 55–56. DOI : <https://doi.org/10.1145/3135932.3135941>
- [61] Fan Long and Martin Rinard. 2016. Automatic patch generation by learning correct code. *SIGPLAN Not.* 51, 1 (Jan. 2016), 298–312. DOI : <https://doi.org/10.1145/2914770.2837617>
- [62] Yunlong Lu, Na Meng, and Wenxin Li. 2021. FAPR: Fast and accurate program repair for introductory programming courses. Retrieved from <http://arxiv.org/abs/2107.06550>
- [63] Andrew Luxton-Reilly, Simon, Ibrahim Albluwi, Brett A. Becker, Michail Giannakos, Amruth N. Kumar, Linda Ott, James Paterson, Michael James Scott, Judy Sheard, and Claudia Szabo. 2018. Introductory programming: A systematic literature review. In *Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE 2018 Companion)*. ACM, New York, NY, 55–106. DOI : <https://doi.org/10.1145/3293881.3295779>
- [64] Matias Martinez and Martin Monperrus. 2016. ASTOR: A program repair library for java (demo). In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA 2016)*. ACM, New York, NY, 441–444. DOI : <https://doi.org/10.1145/2931037.2948705>
- [65] Samiha Marwan, Bitu Akram, Tiffany Barnes, and Thomas W. Price. 2022. Adaptive immediate feedback for block-based programming: Design and evaluation. *IEEE Transactions on Learning Technologies* 15, 3 (2022), 406–420. DOI : <https://doi.org/10.1109/TLT.2022.3180984>
- [66] Samiha Marwan, Ge Gao, Susan Fisk, Thomas W. Price, and Tiffany Barnes. 2020. Adaptive immediate feedback can improve novice programming engagement and intention to persist in computer science. In *Proceedings of the 2020 ACM Conference on International Computing Education Research (ICER '20)*. ACM, New York, NY, 194–203. DOI : <https://doi.org/10.1145/3372782.3406264>
- [67] Samiha Marwan, Nicholas Lytle, Joseph Jay Williams, and Thomas Price. 2019. The impact of adding textual explanations to next-step hints in a novice programming environment. In *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '19)*. ACM, New York, NY, 520–526. DOI : <https://doi.org/10.1145/3304221.3319759>

- [68] Jessica McBroom, Irena Koprsinska, and Kalina Yacef. 2021. A survey of automated programming hint generation: The hints framework. *ACM Computing Surveys* 54, 8 (2021), 1–27. DOI: <https://doi.org/10.1145/3469885>
- [69] Davin McCall and Michael Kölling. 2019. A new look at novice programmer errors. *ACM Transactions on Computing Education* 19, 4 (2019), 1–30. DOI: <https://doi.org/10.1145/3335814>
- [70] Davin McCall and Michael Kölling. 2014. Meaningful categorisation of novice programmer errors. In *Proceedings of the 2014 IEEE Frontiers in Education Conference (FIE) Proceedings*. IEEE, Madrid, Spain, 1–8. DOI: <https://doi.org/10.1109/FIE.2014.7044420>
- [71] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)*. ACM, New York, NY, 691–701. DOI: <https://doi.org/10.1145/2884781.2884807>
- [72] Rodrigo Pessoa Medeiros, Geber Lisboa Ramalho, and Taciana Pontual Falcão. 2018. A systematic literature review on teaching and learning introductory programming in higher education. *IEEE Transactions on Education* 62, 2 (2018), 77–90. DOI: <https://doi.org/10.1109/TE.2018.2864133>
- [73] Marcus Messer, Neil C. C. Brown, Michael Kölling, and Miaojing Shi. 2024. Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education* 24, 1, Article 10 (Feb. 2024), 43 pages. DOI: <https://doi.org/10.1145/3636515>
- [74] Martin Monperrus. 2018. Automatic software repair: A bibliography. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–24. DOI: <https://doi.org/10.1145/3105906>
- [75] Susanne Narciss. 2008. Feedback strategies for interactive learning tasks. In *Handbook of Research on Educational Communications and Technology*. J. Michael Spector, M. David Merrill, Jeroen van Merriënboer, and Marcy P. Driscoll (Eds.), Routledge, Milton Park, 125–143.
- [76] John C. Nesbit, Oluola O. Adesope, Qing Liu, and Wenting Ma. 2014. How effective are intelligent tutoring systems in computer science education? In *Proceedings of the 2014 IEEE 14th International Conference on Advanced Learning Technologies*. IEEE, Athens, Greece, 99–103. DOI: <https://doi.org/10.1109/ICALT.2014.38>
- [77] Florian Obermüller, Ute Heuer, and Gordon Fraser. 2021. Guiding next-step hint generation using automated tests. In *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE '21)*. ACM, New York, NY, 220–226. DOI: <https://doi.org/10.1145/3430665.3456344>
- [78] OpenAI. 2022. Introducing ChatGPT. OpenAI. Retrieved August 21, 2023 from <https://openai.com/blog/chatgpt>
- [79] Eng Lieh Ouh, Benjamin Kok Siew Gan, Kyong Jin Shim, and Swavek Wlodkowski. 2023. ChatGPT, can you generate solutions for my coding exercises? An evaluation on its effectiveness in an undergraduate Java programming course. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE '23)*. ACM, New York, NY, 54–60. DOI: <https://doi.org/10.1145/3587102.3588794>
- [80] José Carlos Paiva, Álvaro Figueira, and José Paulo Leal. 2023. Bibliometric analysis of automated assessment in programming education: A deeper insight into feedback. *Electronics* 12, 10 (2023), 2254. DOI: <https://doi.org/10.3390/electronics12102254>
- [81] José Carlos Paiva, José Paulo Leal, and Álvaro Figueira. 2022. Automated assessment in computer science education: A state-of-the-art review. *ACM Transactions on Computing Education* 22, 3 (2022), 1–40. DOI: <https://doi.org/10.1145/3513140>
- [82] José Carlos Paiva, José Paulo Leal, and Álvaro Figueira. 2023. PROGpedia: Collection of source-code submitted to introductory programming assignments. *Data in Brief* 46 (2023), 108887. DOI: <https://doi.org/10.1016/j.dib.2023.108887>
- [83] Maciej Pankiewicz and Ryan S. Baker. 2023. Large language models (GPT) for automating feedback on programming assignments. In *Proceedings of the 31st International Conference on Computers in Education (ICCE '23)*. Ju-Ling Shih, Akihiro Kashiwara, Weiqin Chen, and Hiroaki Ogata (Eds.), Asia-Pacific Society for Computers in Education, Matsue Shimane JAPAN, 68–77. DOI: <https://doi.org/10.58459/ICCE.2023.950>
- [84] Sagar Parihar, Ziyaan Dadachanji, Praveen Kumar Singh, Rajdeep Das, Amey Karkare, and Arnab Bhattacharya. 2017. Automatic grading and feedback using program repair for introductory programming courses. In *Proceedings of the 2017 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '17)*. ACM, New York, NY, 92–97. DOI: <https://doi.org/10.1145/3059009.3059026>
- [85] Pithchaya Mangpo Phothilimthana and Sumukh Sridhara. 2017. High-Coverage hint generation for massive courses: Do automated hints help CS1 students? In *Proceedings of the 2017 ACM Conference on Innovation and Technology in Computer Science Education (Bologna, Italy) (ITiCSE '17)*. ACM, New York, NY, USA, 182–187. <https://doi.org/10.1145/3059009.3059058>.
- [86] Tung Phung, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generating high-precision feedback for programming syntax errors using large language models. In *Proceedings of the 16th International Conference on Educational Data Mining*. Mingyu Feng, Tanja Käser, and Partha Talukdar (Eds.), International Educational Data Mining Society, Bengaluru, India, 370–377. DOI: <https://doi.org/10.5281/zenodo.8115653>

- [87] Thomas Price, Rui Zhi, and Tiffany Barnes. 2017. Evaluation of a data-driven feedback algorithm for open-ended programming. In *Proceedings of the 10th International Conference on Educational Data Mining*. International Educational Data Mining Society, Wuhan, China, 192–197.
- [88] Thomas W. Price, Yihuan Dong, Rui Zhi, Benjamin Paaßen, Nicholas Lytle, Veronica Cateté, and Tiffany Barnes. 2019. A comparison of the quality of data-driven programming hint generation algorithms. *International Journal of Artificial Intelligence in Education* 29 (2019), 368–395. DOI: <https://doi.org/10.1007/s40593-019-00177-z>
- [89] Thomas W. Price, Samiha Marwan, and Joseph Jay Williams. 2021. Exploring design choices in data-driven hints for python programming homework. In *Proceedings of the 8th ACM Conference on Learning @ Scale (L@S '21)*. ACM, New York, NY, 283–286. DOI: <https://doi.org/10.1145/3430895.3460159>
- [90] Kelly Rivers and Kenneth R. Koedinger. 2017. Data-driven hint generation in vast solution spaces: A self-improving python programming tutor. *International Journal of Artificial Intelligence in Education* 27, 1 (2017), 37–64.
- [91] Anthony V. Robins. 2019. Novice programmers and introductory programming. In *The Cambridge Handbook of Computing Education Research*. Anthony V. Robins and Sally A. Fincher (Eds.), Cambridge University Press, Cambridge, 327–376. DOI: <https://doi.org/10.1017/9781108654555.013>
- [92] Hemilis Joyse Barbosa Rocha, Patricia Cabral De Azevedo Restelli Tedesco, and Evandro De Barros Costa. 2022. On the use of feedback in learning computer programming by novices: A systematic literature mapping. *Informatics in Education* 22, 2 (2022), 209–232. DOI: <https://doi.org/10.15388/infedu.2023.09>
- [93] Rohaida Romli, Shahida Sulaiman, and Kamal Zuhairi Zamli. 2010. Automatic programming assessment and test data generation a review on its approaches. In *Proceedings of the 2010 International Symposium on Information Technology*, Vol. 3, 1186–1192. Retrieved from <https://api.semanticscholar.org/CorpusID:18222797>
- [94] Georgios Sakkas, Madeline Endres, Benjamin Cosman, Westley Weimer, and Ranjit Jhala. 2020. Type error feedback via analytic program repair. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2020)*. ACM, New York, NY, 16–30. DOI: <https://doi.org/10.1145/3385412.3386005>
- [95] Eddie Antonio Santos, Joshua Charles Campbell, Dhvani Patel, Abram Hindle, and José Nelson Amaral. 2018. Syntax and sensibility: Using language models to detect and correct syntax errors. In *Proceedings of the 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, Campobasso, Italy, 311–322. DOI: <https://doi.org/10.1109/SANER.2018.8330219>
- [96] Eddie Antonio Santos, Prajish Prasad, and Brett A. Becker. 2023. Always provide context: The effects of code context on programming error message enhancement. In *Proceedings of the ACM Conference on Global Computing Education Vol 1 (CompEd '23)*. ACM, New York, NY, 147–153. DOI: <https://doi.org/10.1145/3576882.3617909>
- [97] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An analysis of the automatic bug fixing performance of ChatGPT. In *Proceedings of the 2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE Computer Society, Los Alamitos, CA, 23–30. DOI: <https://doi.org/10.1109/APR59189.2023.00012>
- [98] Dowon Song, Woosuk Lee, and Hakjoo Oh. 2021. Context-aware and data-driven feedback generation for programming assignments. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, New York, NY, 328–340. DOI: <https://doi.org/10.1145/3468264.3468598>
- [99] Draylson M. Souza, Katia R. Felizardo, and Ellen F. Barbosa. 2016. A systematic literature review of assessment tools for programming assignments. In *Proceedings of the 2016 IEEE 29th International Conference on Software Engineering Education and Training (CSEET)*. IEEE, Dallas, TX, 147–156. DOI: <https://doi.org/10.1109/CSEET.2016.48>
- [100] John Stamper, Tiffany Barnes, Lorrie Lehmann, and Marvin Croy. 2008. The hint factory: Automatic generation of contextualized help for existing computer aided instruction. In *Proceedings of the 9th International Conference on Intelligent Tutoring Systems Young Researchers Track*. Montreal, Canada, 71–78. Retrieved from <https://dev.stamper.org/publications/TTS2008Stamper.pdf>
- [101] Andrew Taylor, Alexandra Vassar, Jake Renzella, and Hammond Pearce. 2023. Dcc-help: Generating context-aware compiler error explanations with large language models. arXiv:2308.11873. Retrieved from <https://arxiv.org/abs/2308.11873>
- [102] Andrew Taylor, Alexandra Vassar, Jake Renzella, and Hammond Pearce. 2024. Dcc-help: Transforming the role of the compiler by generating context-aware error explanations with large language models. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE '24)*. ACM, New York, NY, 1314–1320. DOI: <https://doi.org/10.1145/3626252.3630822>
- [103] Daniel Toll, Anna Wingkvist, and Morgan Ericsson. 2020. Current state and next steps on automated hints for students learning to code. In *Proceedings of the 2020 IEEE Frontiers in Education Conference (FIE)*. IEEE, Uppsala, Sweden, 1–5. DOI: <https://doi.org/10.1109/FIE44824.2020.9274053>
- [104] Pauliina Tuomi, Jari Multisilta, Petri Saarikoski, and Jaakko Suominen. 2018. Coding skills as a success factor for a society. *Education and Information Technologies* 23, 1 (Jan. 2018), 419–434. DOI: <https://doi.org/10.1007/s10639-017-9611-4>

- [105] Kurt VanLehn. 2011. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist* 46, 4 (2011), 197–221. DOI : <https://doi.org/10.1080/00461520.2011.611369>
- [106] Pierpaolo Vittorini and Alessandra Galassi. 2023. rDSA: An intelligent tool for data science assignments. *Multimedia Tools and Applications* 82, 9 (2023), 12879–12905. DOI : <https://doi.org/10.1007/s11042-022-14053-x>
- [107] Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, align, and repair: Data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. ACM, New York, NY, 481–495. DOI : <https://doi.org/10.1145/3192366.3192384>
- [108] Wengran Wang, Yudong Rao, Rui Zhi, Samiha Marwan, Ge Gao, and Thomas W. Price. 2020. Step tutor: Supporting students through step-by-step example-based feedback. In *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '20)*. ACM, New York, NY, 391–397. DOI : <https://doi.org/10.1145/3341525.3387411>
- [109] Christopher Watson and Frederick W. B. Li. 2014. Failure rates in introductory programming revisited. In *Proceedings of the 2014 Conference on Innovation & Technology in Computer Science Education (ITiCSE '14)*. ACM, New York, NY, 39–44. DOI : <https://doi.org/10.1145/2591708.2591749>
- [110] Westley Weimer, Zachary P. Fry, and Stephanie Forrest. 2013. Leveraging program equivalence for adaptive program repair: Models and first results. In *Proceedings of the 2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Silicon Valley, CA, 356–366. DOI : <https://doi.org/10.1109/ASE.2013.6693094>
- [111] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, Melbourne, Australia, 1482–1494. DOI : <https://doi.org/10.1109/ICSE48619.2023.00129>
- [112] Chunqiu Steven Xia and Lingming Zhang. 2023. Conversational automated program repair. arXiv:2301.13246. Retrieved from <https://arxiv.org/abs/2301.13246>
- [113] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*. Maria Christakis and Michael Pradel (Eds.), ACM, New York, NY, 819–831. DOI : <https://doi.org/10.1145/3650212.3680323>
- [114] He Ye and Martin Monperrus. 2024. ITER: Iterative neural repair for multi-location patches. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 10, 13 pages. DOI : <https://doi.org/10.1145/3597503.3623337>
- [115] Jooyong Yi, Umair Z. Ahmed, Amey Karkare, Shin Hwei Tan, and Abhik Roychoudhury. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017)*. ACM, New York, NY, 740–751. DOI : <https://doi.org/10.1145/3106237.3106262>
- [116] Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing bugs in python assignments using large language models. arXiv:2209.14876. Retrieved from <https://arxiv.org/abs/2209.14876>
- [117] Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. PyDex: Repairing bugs in introductory python assignments using LLMs. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 1100–1124. DOI : <https://doi.org/10.1145/3649850>
- [118] Kurtis Zimmerman and Chandan R. Rupakheti. 2015. An automated framework for recommending program elements to novices. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE '15)*. IEEE Press, Lincoln, NE, 283–288. DOI : <https://doi.org/10.1109/ASE.2015.54>

Received 1 September 2023; revised 4 July 2025; accepted 31 July 2025